VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

ANNAPURNA MUGALI(1BM24CS046)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026

CERTIFICATE

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

This is to certify that the Lab work entitled **"ANALYSIS AND DESIGN OF ALGORITHMS"** carried out by **ANNAPURNA MUGALI (1BM24CS046)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Kanchana M Dixit
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

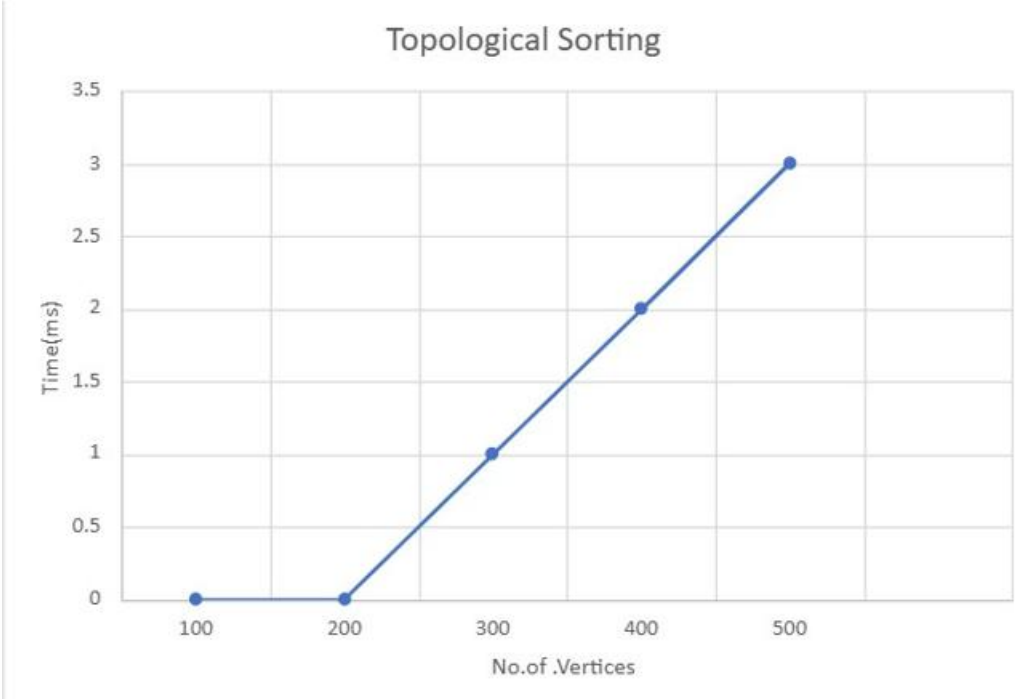
Write program to obtain the Topological ordering of vertices in a given digraph

```
#include <stdio.h>
#define MAX 100
int main() {
   int n, V[MAX][MAX];
   int indegree[MAX] = {0};
   int visited[MAX] = {0};
   int TP[MAX];
   int tp_count = 0;
   printf("Enter number of vertices: ");
   scanf("%d", &n);
   printf("Enter adjacency matrix:\n");
   for(int i = 0; i < n; i++) {
      for(int j = 0; j < n; j++) {
         scanf("%d", &V[i][j]);
      }
   }
   for(int i = 0; i < n; i++) {
      indegree[i] = 0;
      for(int j = 0; j < n; j++) {
         if(V[j][i] == 1) {
            indegree[i]++;
         }
      }
   }
   while(1) {
      int w = -1;
```

```
        for(int i = 0; i < n; i++) {
            if(visited[i] == 0 && indegree[i] == 0) {
                w = i;
                break;
            }
        }

        if(w == -1)
            break;
        TP[tp_count++] = w;
        visited[w] = 1;
        for(int i = 0; i < n; i++) {
            if(V[w][i] == 1) {
                indegree[i]--;
            }
        }
    }
    if(tp_count < n) {
        printf("No Topological Sequence (cycle detected)\n");
    } else {
        printf("Topological Order:\n");
        for(int i = 0; i < tp_count; i++) {
            printf("%d ", TP[i]);
        }
    }

    return 0;
}
```

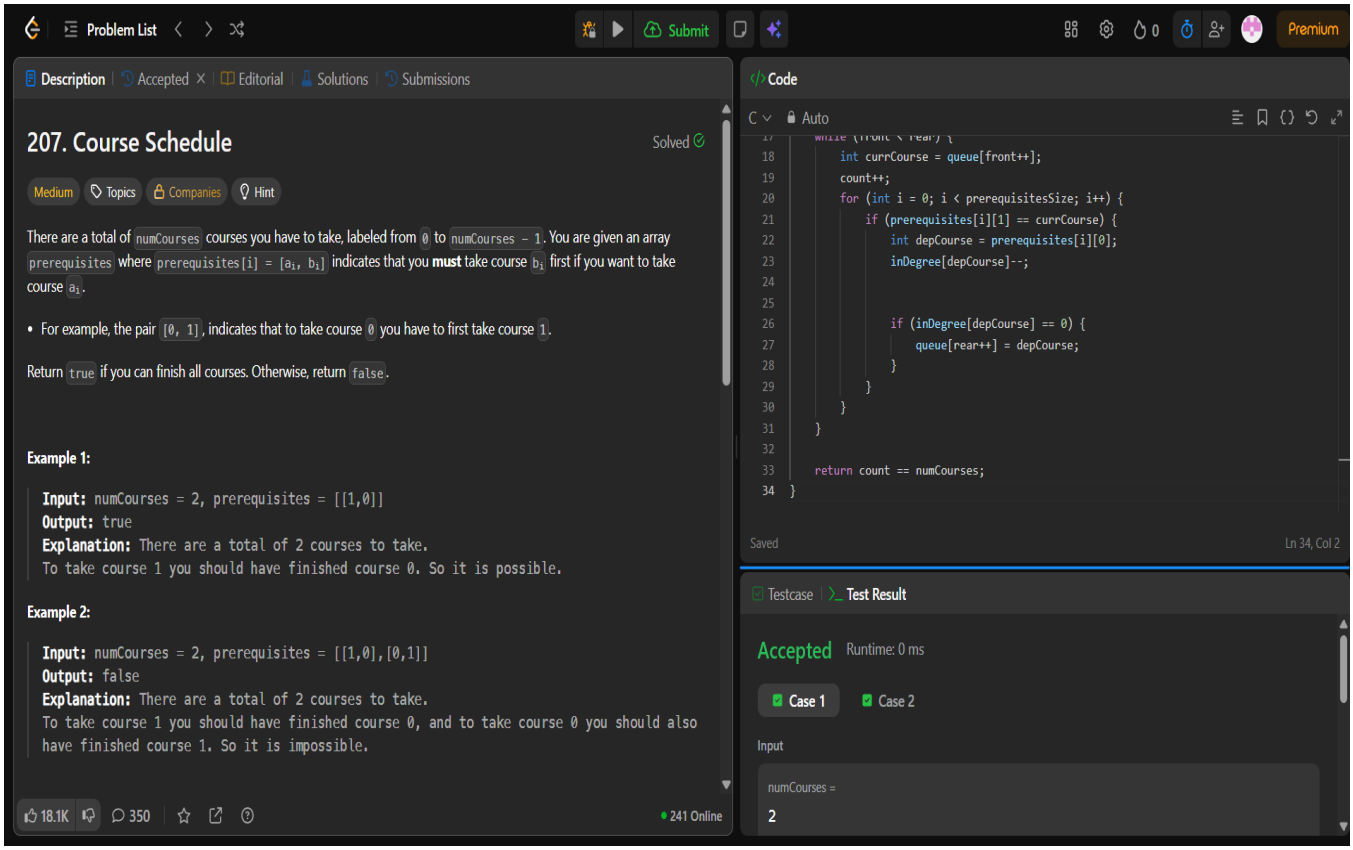
Output:

```
Enter number of vertices: 5
Enter adjacency matrix:
0 1 0 0 0
0 0 1 0 0
0 0 0 1 0
0 0 0 0 1
0 0 0 0 0
Topological Order:
0 1 2 3 4
Process returned 0 (0x0)   execution time : 79.900 s
Press any key to continue.
```



LEETCODE-207-Course Schedule

```
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int* prerequisitesColSize) {
    int inDegree[2000] = {0};
    int queue[2000], front, rear, count;
    front = rear = count = 0;
    for (int i = 0; i < prerequisitesSize; i++) {
        int course = prerequisites[i][0];
        inDegree[course]++;
    }
    for (int i = 0; i < numCourses; i++) {
        if (inDegree[i] == 0) {
            queue[rear++] = i;
        }
    }
    while (front < rear) {
        int currCourse = queue[front++];
        count++;
        for (int i = 0; i < prerequisitesSize; i++) {
            if (prerequisites[i][1] == currCourse) {
                int depCourse = prerequisites[i][0];
                inDegree[depCourse]--;
                if (inDegree[depCourse] == 0) {
                    queue[rear++] = depCourse;
                }
            }
        }
    }
    return count == numCourses;
}
```


Problem List
Submit
Premium
Description
Accepted
Editorial
Solutions
Submissions
207. Course Schedule
Solved
Medium
Topics
Companies
Hint
There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [ai, bi] indicates that you must take course bi first if you want to take course ai.
• For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1.
Return true if you can finish all courses. Otherwise, return false.
Example 1:
Input: numCourses = 2, prerequisites = [[1,0]]
Output: true
Explanation: There are a total of 2 courses to take.
To take course 1 you should have finished course 0. So it is possible.
Example 2:
Input: numCourses = 2, prerequisites = [[1,0],[0,1]]
Output: false
Explanation: There are a total of 2 courses to take.
To take course 1 you should have finished course 0, and to take course 0 you should also have finished course 1. So it is impossible.
18.1K
350
241 Online
Code
C
Auto
int currCourse = queue[front++];
count++;
for (int i = 0; i < prerequisitesSize; i++) {
if (prerequisites[i][1] == currCourse) {
int depCourse = prerequisites[i][0];
inDegree[depCourse]--;
if (inDegree[depCourse] == 0) {
queue[rear++] = depCourse;
}
}
}
}
return count == numCourses;
}
Saved
Ln 34, Col 2
Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Input
numCourses =
2

LAB PROGRAM 2:

Implement Johnson Trotter algorithm to generate permutations.

```
#define LEFT_TO_RIGHT true
#define RIGHT_TO_LEFT false


void swap(int *a, int *b) {
   int temp = *a;
   *a = *b;
   *b = temp;
}
int getMobile(int a[], bool dir[], int n) {
   int mobile_prev = 0, mobile = 0;
   for (int i = 0; i < n; i++) {
      if (dir[i] == RIGHT_TO_LEFT && i != 0) {
         if (a[i] > a[i - 1] && a[i] > mobile_prev) {
            mobile = i + 1;
            mobile_prev = a[i];
         }
      }
      if (dir[i] == LEFT_TO_RIGHT && i != n - 1) {
         if (a[i] > a[i + 1] && a[i] > mobile_prev) {
            mobile = i + 1;
            mobile_prev = a[i];
         }
      }
   }
   return mobile;
}
```

```
void printArray(int a[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}
void printPermutation(int n) {
    int a[n];
    bool dir[n];
    for (int i = 0; i < n; i++) {
        a[i] = i + 1;
        dir[i] = RIGHT_TO_LEFT;
    }
     printArray(a, n);
     int mobile;
     while ((mobile = getMobile(a, dir, n)) != 0) {
        int pos = mobile - 1;
        int mobile_value = a[pos];
        if (dir[pos] == RIGHT_TO_LEFT) {
            swap(&a[pos], &a[pos - 1]);
            bool temp = dir[pos];
            dir[pos] = dir[pos - 1];
            dir[pos - 1] = temp;
            pos--;
        } else {
            swap(&a[pos], &a[pos + 1]);
            bool temp = dir[pos];
            dir[pos] = dir[pos + 1];
```

```
            dir[pos + 1] = temp;
            pos++;
        }
        for (int i = 0; i < n; i++) {
            if (a[i] > mobile_value) {
                dir[i] = !dir[i];
            }
        }
        printArray(a, n);
    }
}

int main() {
    int n;
    printf("Enter value of n: ");
    scanf("%d", &n);

    printPermutation(n);
    return 0;
}
```

Output:

```
Enter value of n: 3
1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3

Process returned 0 (0x0)   execution time : 1.820 s
Press any key to continue.
```

Lab Program-3:

Sort a given set of N integer elements using merge sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort

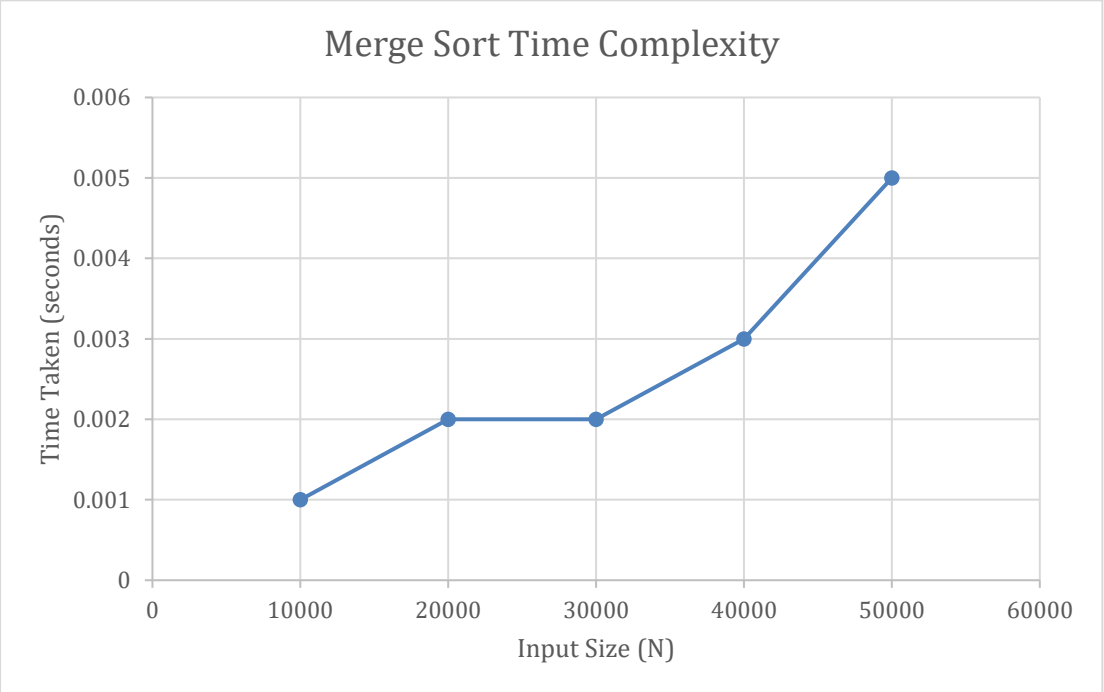
```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAX 50000
int arr[MAX], temp[MAX];
void merge(int low, int mid, int high)
{
   int i = low;
   int j = mid + 1;
   int k = low;
   while (i <= mid && j <= high)
   {
      if (arr[i] < arr[j])
         temp[k++] = arr[i++];
      else
         temp[k++] = arr[j++];
   }

   while (i <= mid)
      temp[k++] = arr[i++];

   while (j <= high)
      temp[k++] = arr[j++];

   for (i = low; i <= high; i++)
```

```
        arr[i] = temp[i];
}
void mergesort(int low, int high)
{
    if (low < high)
    {
        int mid = (low + high) / 2;
        mergesort(low, mid);
        mergesort(mid + 1, high);
        merge(low, mid, high);
    }
}
int main()
{
    int n, i;
    clock_t start, end;
    double time_taken;

    srand(time(NULL));

    printf("N\tTime Taken (seconds)\n");

    for (n = 10000; n <= 50000; n += 10000)
    {
        for (i = 0; i < n; i++)
            arr[i] = rand() % 50000;

        start = clock();
```

```
        mergesort(0, n - 1);

        end = clock();
        time_taken = (double)(end - start) / CLOCKS_PER_SEC;
        printf("%d\t%.10f\n", n, time_taken);
    }
    return 0;
}
```

Output:

```
N       Time Taken (seconds)
10000   0.0010000000
20000   0.0010000000
30000   0.0020000000
40000   0.0030000000
50000   0.0040000000

Process returned 0 (0x0)   execution time : 3.828 s
Press any key to continue.
```



LAB PROGRAM 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

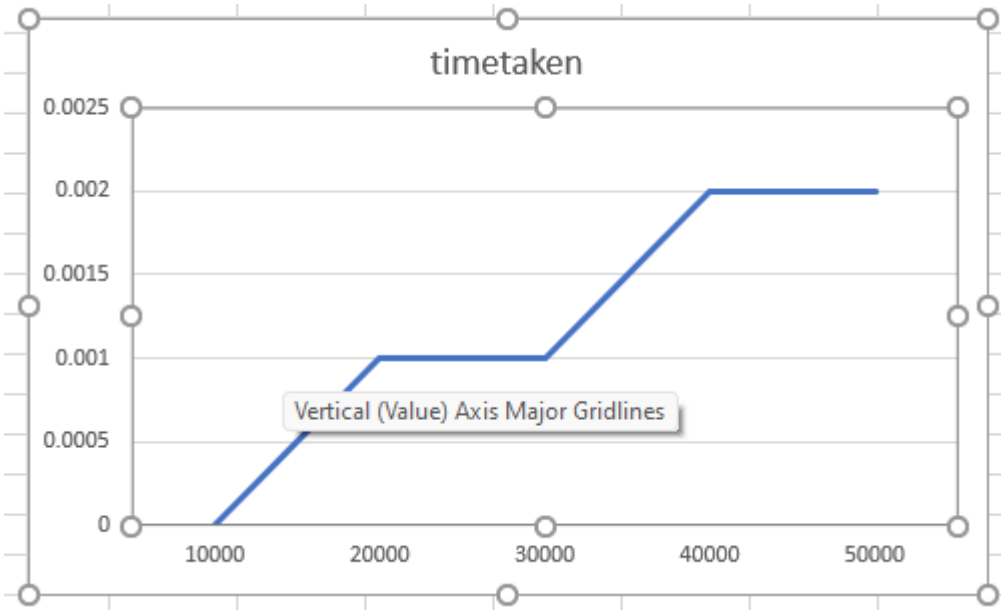
```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAX 50000
int arr[MAX];
int partition(int a[], int low, int high) {
   int pivot = a[low];
   int i = low;
   int j = high + 1;

   while (1) {
      do {
         i++;
      } while (i <= high && a[i] <= pivot);

      do {
         j--;
      } while (a[j] > pivot);

      if (i >= j)
         break;

      int temp = a[i];
      a[i] = a[j];
      a[j] = temp;
   }
```

```
    int temp = a[low];
    a[low] = a[j];
    a[j] = temp;

    return j;
}
void quickSort(int a[], int low, int high) {
    if (low < high) {
        int j = partition(a, low, high);
        quickSort(a, low, j - 1);
        quickSort(a, j + 1, high);
    }
}

int main() {
    int n, i;
    clock_t start, end;
    double time_taken;

    srand(time(NULL));

    for (n = 10000; n <= 50000; n += 10000) {
        for (i = 0; i < n; i++) {
            arr[i] = rand() % 50000;
        }

        start = clock();
        quickSort(arr, 0, n - 1);
```

```
        end = clock();

        time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;

        printf("n = %d, Time taken = %f seconds\n", n, time_taken);
    }

    return 0;
}
```

Output:

```
n = 10000, Time taken = 0.000000 seconds
n = 20000, Time taken = 0.002000 seconds
n = 30000, Time taken = 0.002000 seconds
n = 40000, Time taken = 0.002000 seconds
n = 50000, Time taken = 0.002000 seconds

Process returned 0 (0x0)   execution time : 0.020 s
Press any key to continue.
```



LAB PROGRAM 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

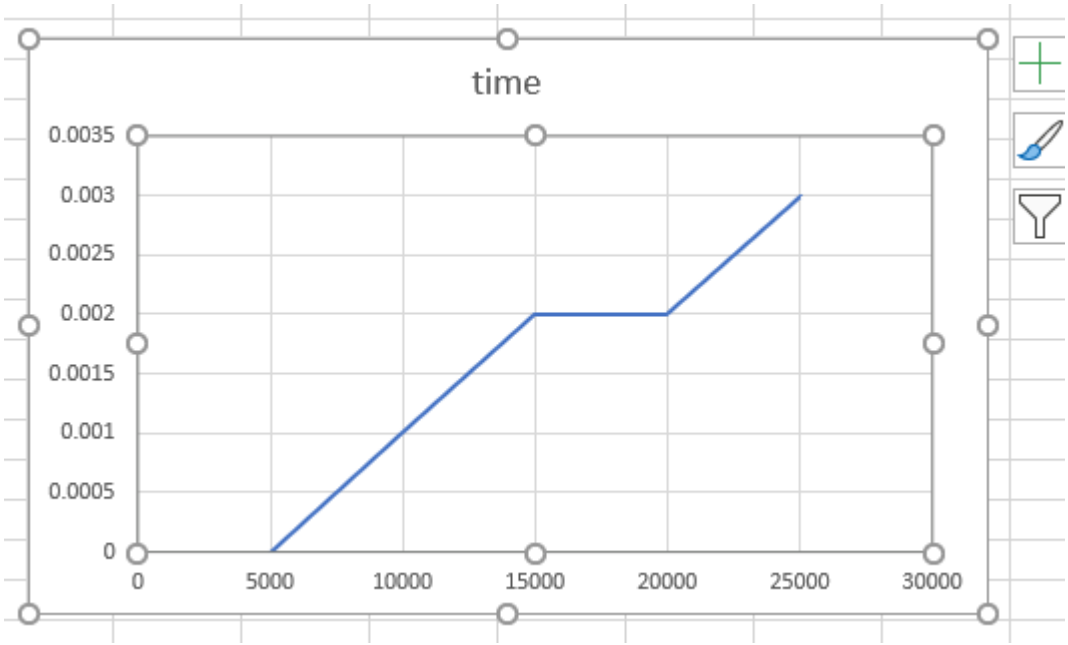
```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        heapify(arr, n, largest);
    }
}
void heapSort(int arr[], int n) {
    for (int i = n/2 - 1; i >= 0; i--)
        heapify(arr, n, i);
```

```
    for (int i = n - 1; i > 0; i--) {
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;

        heapify(arr, i, 0);
    }
}

int main() {
    int sizes[5] = {5000, 10000, 15000, 20000, 25000};

    srand(time(0));

    printf("Size\tTime (seconds)\n");

    for (int s = 0; s < 5; s++) {
        int n = sizes[s];
        int *arr = (int *)malloc(n * sizeof(int));
        for (int i = 0; i < n; i++)
            arr[i] = rand() % 100000;

        clock_t start = clock();
        heapSort(arr, n);
        clock_t end = clock();
        double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
        printf("%d\t%f\n", n, time_taken);
        free(arr);
```

```
    }
    return 0;
}
```

Output:

```
Size    Time (seconds)
5000    0.000000
10000   0.001000
15000   0.002000
20000   0.002000
25000   0.003000

Process returned 0 (0x0)   execution time : 0.016 s
Press any key to continue.
```



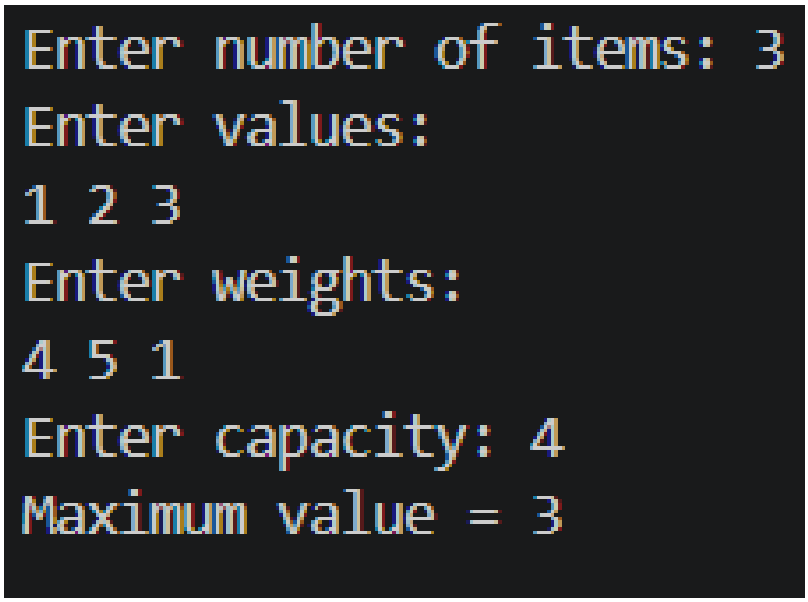
LAB PROGRAM 6:

Implement 0/1 Knapsack problem using dynamic programming.

```
#include <stdio.h>
int max(int a, int b) {
   return (a > b) ? a : b;
}
int zeroOrOne_Knapsack(int n, int val[], int w[], int c)
{
   int a[50][50];
   for (int i = 0; i <= n; i++)
      a[i][0] = 0;
   for (int j = 0; j <= c; j++)
      a[0][j] = 0;
   for (int i = 1; i <= n; i++) {
      for (int j = 1; j <= c; j++) {
         if (w[i-1] > j)
            a[i][j] = a[i-1][j];
         else
            a[i][j] = max(
               a[i-1][j],
               val[i-1] + a[i-1][j - w[i-1]]
            );
      }
   }
   return a[n][c];
}
int main()
{
   int val[20], w[20], c, n;
```

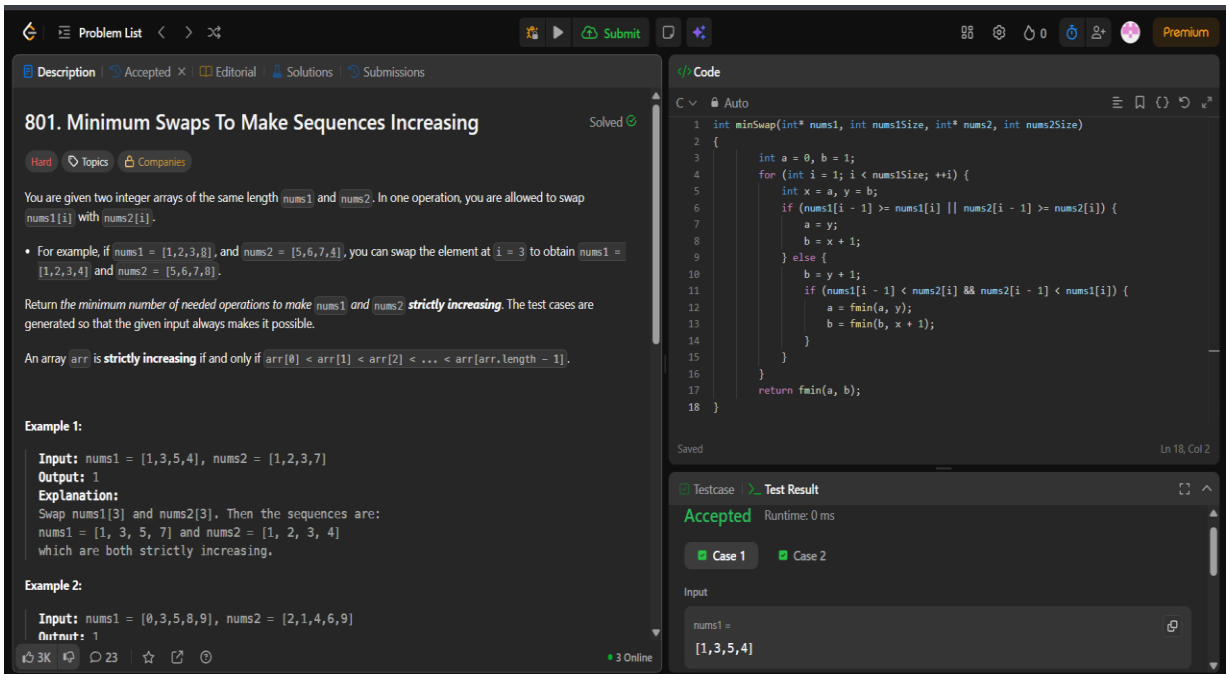
```
    printf("Enter number of items: ");
    scanf("%d", &n);
    printf("Enter values:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &val[i]);
    }
    printf("Enter weights:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &w[i]);
    }
    printf("Enter capacity: ");
    scanf("%d", &c);
    int result = zeroOrOne_Knapsack(n, val, w, c);
    printf("Maximum value = %d\n", result);
    return 0;
}
```

Output:



LEET CODE:801-Minimun Swaps to make Sequences Increasing

```
int minSwap(int* nums1, int nums1Size, int* nums2, int nums2Size)
{     int a = 0, b = 1;
      for (int i = 1; i < nums1Size; ++i) {
         int x = a, y = b;
         if (nums1[i - 1] >= nums1[i] || nums2[i - 1] >= nums2[i]) {
            a = y;
            b = x + 1;
         } else {
            b = y + 1;
            if (nums1[i - 1] < nums2[i] && nums2[i - 1] < nums1[i]) {
               a = fmin(a, y);
               b = fmin(b, x + 1);
               }
            }
         }
      return fmin(a, b);
}
```



LAB PROGRAM 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

```
#include <stdio.h>
int a[10][10], D[10][10], n;
void floyd(int [][10], int);
int min(int, int);
int main()
{
   printf("Enter the no. of vertices:");
   scanf("%d", &n);
   printf("Enter the cost adjacency matrix:\n");
   int i, j;
   for (i = 0; i < n; i++) {
      for (j = 0; j < n; j++) {
         scanf("%d", &a[i][j]);
      }
   }
   floyd(a, n);
   printf("Distance Matrix:\n");
   for (i = 0; i < n; i++) {
      for (j = 0; j < n; j++) {
         printf("%d ", D[i][j]);
      }
      printf("\n");
   }
   return 0;
}
void floyd(int a[][10], int n){
```

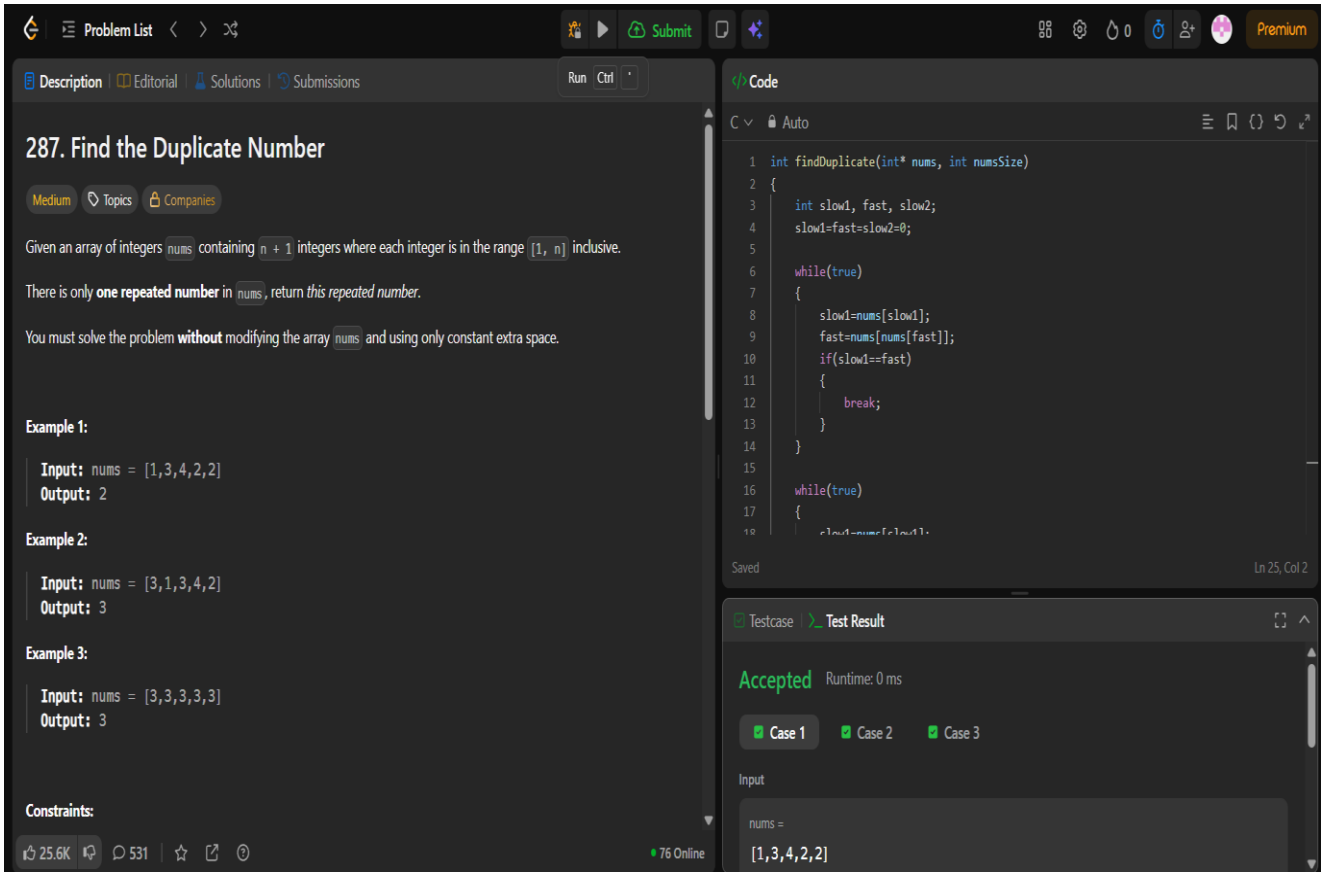
```
int i, j, k;
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            D[i][j] = a[i][j];
        }
    }
    for (k = 0; k < n; k++) {
        for (i = 0; i < n; i++) {
            for (j = 0; j < n; j++) {
                D[i][j] = min(D[i][j], (D[i][k] + D[k][j]));
            }
        }
    }
}
int min(int a, int b)
{
    if (a < b) {
        return a;
    } else {
        return b;
}   }
```

Output:

```
Enter the no. of vertices:4
Enter the cost adjacency matrix:
0 99 3 99
2 0 99 99
99 6 0 1
7 99 9 0
Distance Matrix:
0 9 3 4
2 0 5 6
8 6 0 1
7 15 9 0
```

LEETCODE-287-Find the Duplicate Number

```
int findDuplicate(int* nums, int numsSize)
{
   int slow1, fast, slow2;
   slow1=fast=slow2=0;

   while(true)
   {
      slow1=nums[slow1];
      fast=nums[nums[fast]];
      if(slow1==fast)
      {
         break;
      }
   }

   while(true)
   {
      slow1=nums[slow1];
      slow2=nums[slow2];
      if(slow1==slow2)
      {
            return slow2;
      }
   }
}
```


Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
Run Ctrl '
287. Find the Duplicate Number
Medium
Topics
Companies
Given an array of integers nums containing n + 1 integers where each integer is in the range [1, n] inclusive.
There is only one repeated number in nums, return this repeated number.
You must solve the problem without modifying the array nums and using only constant extra space.
Example 1:
Input: nums = [1,3,4,2,2]
Output: 2
Example 2:
Input: nums = [3,1,3,4,2]
Output: 3
Example 3:
Input: nums = [3,3,3,3,3]
Output: 3
Constraints:
25.6K
531
76 Online
Code
C
Auto
int findDuplicate(int* nums, int numsSize)
{
int slow1, fast, slow2;
slow1=fast=slow2=0;
while(true)
{
slow1=nums[slow1];
fast=nums[nums[fast]];
if(slow1==fast)
{
break;
}
}
while(true)
{
Saved
Ln 25, Col 2
Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
nums =
[1,3,4,2,2]

LAB PROGRAM 8:

a)Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

```
#include <stdio.h>

#define MAX 10
#define INF 9999

int main() {
   int n, cost[MAX][MAX], visited[MAX] = {0};
   int i, j, ne = 1;
   int min, mincost = 0;
   int a = 0, b = 0;

   printf("Enter number of vertices: ");
   scanf("%d", &n);

   printf("Enter the cost adjacency matrix:\n");
   for(i = 0; i < n; i++) {
      for(j = 0; j < n; j++) {
         scanf("%d", &cost[i][j]);

         if(cost[i][j] == 0)
            cost[i][j] = INF;
      }
   }

   visited[0] = 1;

   printf("\nEdges in MST:\n");

   while(ne < n) {
```

```
        min = INF;

        for(i = 0; i < n; i++) {
           if(visited[i]) {
              for(j = 0; j < n; j++) {
                 if(!visited[j] && cost[i][j] < min) {
                    min = cost[i][j];
                    a = i;
                    b = j;
                 }
              }
           }
        }

        printf("%d -> %d = %d\n", a, b, min);

        visited[b] = 1;
        mincost += min;
        ne++;
     }

     printf("\nMinimum cost of spanning tree = %d\n", mincost);

     return 0;
}
```

Output:

```
Enter number of vertices: 5
Enter the cost adjacency matrix:
0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
0 8 0 0 9
0 5 7 9 0

Edges in MST:
0 -> 1 = 2
1 -> 2 = 3
1 -> 4 = 5
0 -> 3 = 6

Minimum cost of spanning tree = 16
```

b. Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 30
typedef struct {
   int u, v, w;
} Edge;
Edge edges[MAX], result[MAX];
int parent[MAX];
int find(int i) {
   while (parent[i] != i)
      i = parent[i];
   return i;
}
void unionSet(int i, int j) {
   parent[i] = j;
}
void sortEdges(int e) {
   int i, j;
   Edge temp;
   for (i = 0; i < e - 1; i++) {
      for (j = 0; j < e - i - 1; j++) {
         if (edges[j].w > edges[j + 1].w) {
            temp = edges[j];
            edges[j] = edges[j + 1];
            edges[j + 1] = temp;
         }
      }
   }
```

```
}
int main() {
    int n, e, i, j, u, v;
    int count = 0, cost = 0;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter number of edges: ");
    scanf("%d", &e);
    printf("Enter edges (u v weight):\n");
    for (i = 0; i < e; i++) {
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
    }
    for (i = 0; i < n; i++)
        parent[i] = i;
    sortEdges(e);
    for (i = 0; i < e && count < n - 1; i++) {
        u = find(edges[i].u);
        v = find(edges[i].v);
        if (u != v) {
            result[count++] = edges[i];
            cost += edges[i].w;
            unionSet(u, v);
        }
    }
    printf("\nMinimum Spanning Tree:\n");
    for (i = 0; i < count; i++) {
        printf("%d -- %d == %d\n", result[i].u, result[i].v, result[i].w);
    }
    printf("Total cost = %d\n", cost);
```

```
    return 0;
}
```

Output:

```
Enter number of vertices: 7
Enter number of edges: 9
Enter edges (u v weight):
1 6 10
6 5 26
5 4 22
4 3 12
3 2 16
2 1 28
5 7 24
7 2 14
7 4 18

Minimum Spanning Tree:
1 -- 6 == 10
4 -- 3 == 12
7 -- 2 == 14
3 -- 2 == 16
5 -- 4 == 22
6 -- 5 == 26
Total cost = 100
```

Lab program 9:

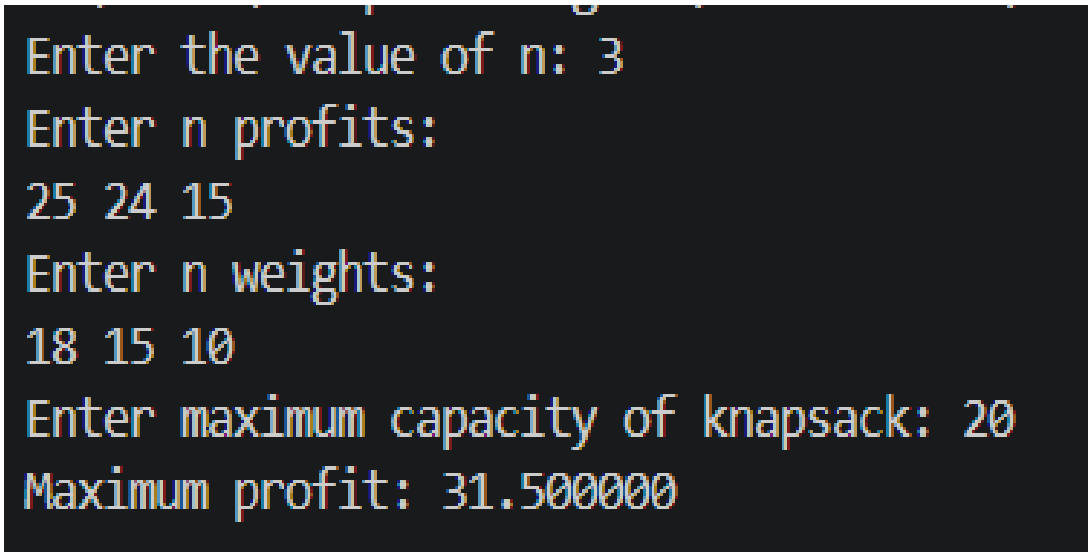
Implement Fractional Knapsack using Greedy technique.

```
#include <stdio.h>


int n, m;
float x[100], p[100], w[100];


void sort() {
   for(int i = 0; i < n - 1; i++) {
      for(int j = i + 1; j < n; j++) {
         if((p[i] / w[i]) < (p[j] / w[j])) {
            float temp1,temp2;


            temp1 = p[i];
            p[i] = p[j];
            p[j] = temp1;


            temp2 = w[i];
            w[i] = w[j];
            w[j] = temp2;
         }
      }
   }
}


float greedy_ks(int m, int n) {
   int i;
```

```
    for(i = 0; i < n; i++) {
        x[i] = 0;
    }

    float rem_cap = m;

    for(i = 0; i < n; i++) {
        if(w[i] > rem_cap)
            break;

        x[i] = 1.0;
        rem_cap = rem_cap - w[i];
    }

    if(i < n) {
        x[i] = rem_cap / w[i];
    }

    float sum = 0;
    for(i = 0; i < n; i++) {
        sum += p[i] * x[i];
    }

    return sum;
}

int main() {
    printf("Enter the value of n: ");
```

```
    scanf("%d", &n);

    printf("Enter n profits:\n");
    for(int i = 0; i < n; i++) {
        scanf("%f", &p[i]);
    }
    printf("Enter n weights:\n");
    for(int i = 0; i < n; i++) {
        scanf("%f", &w[i]);
    }
    printf("Enter maximum capacity of knapsack: ");
    scanf("%d", &m);
    sort();
    float sum = greedy_ks(m, n);
    printf("Maximum profit: %f\n", sum);
    return 0;
}
```

Output:



Lab program 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include <stdio.h>

#define INF 99999

int V;

int minDistance(int dist[], int visited[]) {
   int min = INF, min_index;


   for (int i = 0; i < V; i++) {
      if (visited[i] == 0 && dist[i] <= min) {
         min = dist[i];
         min_index = i;
      }
   }
   return min_index;
}


void dijkstra(int graph[V][V], int src) {
   int dist[V];
   int visited[V];


   for (int i = 0; i < V; i++) {
      dist[i] = INF;
      visited[i] = 0;
   }


   dist[src] = 0;


   for (int count = 0; count < V - 1; count++) {
      int u = minDistance(dist, visited);
```

```
        visited[u] = 1;

        for (int v = 0; v < V; v++) {
            if (!visited[v] &&
                graph[u][v] &&
                dist[u] != INF &&
                dist[u] + graph[u][v] < dist[v]) {

                dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    printf("Vertex \t Distance from Source\n");
    for (int i = 0; i < V; i++) {
        printf("%d \t\t %d\n", i, dist[i]);
    }
}

int main() {
    int graph[20][20];
    printf("enter number of points:");
    scanf("%d",&V);
    printf("enter distance matrix:");
    for(int i=0;i<V;i++){
        for(int j=0;j<V                ;j++){
            scanf("%d",&graph[i][j]);
        }
    }

    int source = 0;
```

```
    dijkstra(graph, source);

    return 0;
}
```

Output:

```
enter number of points:5
enter distance matrix:0 10 0 30 100
10 0 50 0 0
0 50 0 20 10
30 0 20 0 60
100 0 10 60 0
Vertex   Distance from Source
0                0
1                10
2                150
3                30
4                100
```

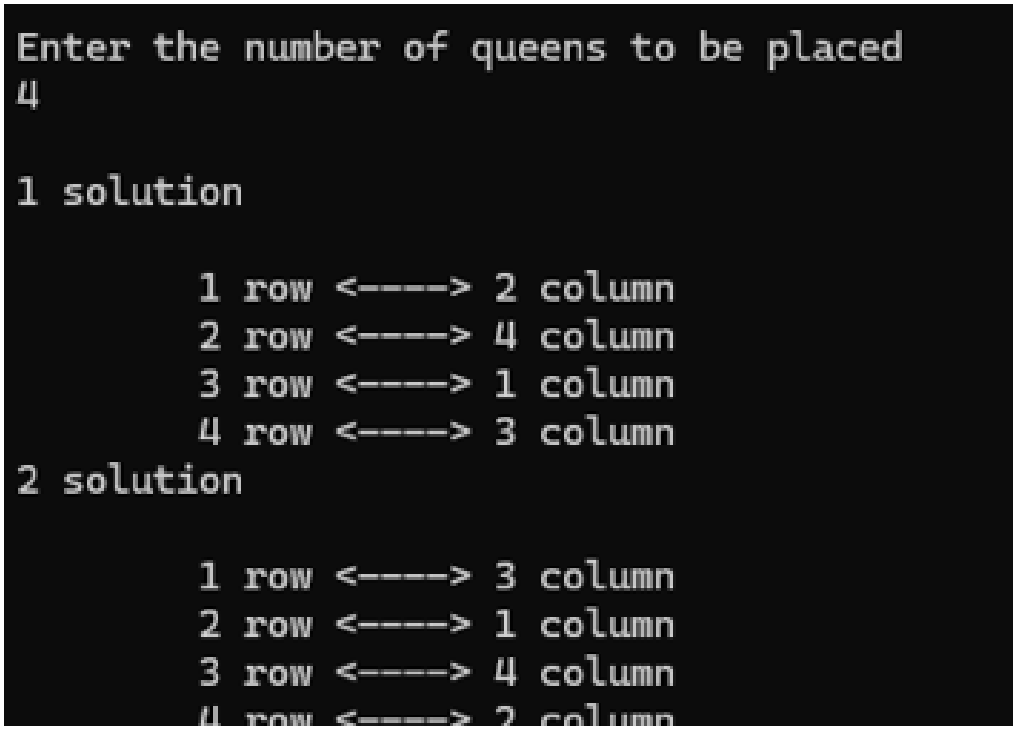
Lab program 11:

Implement "N-Queens Problem" using Backtracking.

```
#include<stdio.h>
#include<conio.h>
#include<math.h>
int x[20], count = 1;
void queens(int, int);
int place(int, int);
int main()
{
   int n, k = 1;
   printf("\nEnter the number of queens to be placed\n");
   scanf("%d", &n);
   queens(k, n);
   getch();
}

void queens(int k, int n)
{
   int i, j;
   for(j = 1; j <= n; j++)
   {
      if(place(k, j))
      {
         x[k] = j;
         if(k == n)
         {
            printf("\n%d solution\n", count);
```

```
            count++;
            for(i = 1; i <= n; i++)
            {
               printf("\n\t%d row <----> %d column", i, x[i]);
            }
             getch();
         }
         else
         {
            queens(k + 1, n);
         }
      }
   }
}

int place(int k, int j)
{
   int i;
   for(i = 1; i < k; i++)
   {
      if((x[i] == j) || (abs(x[i] - j) == abs(i - k)))
      {
         return 0;
      }
   } return 1;
}
```

Output:


Enter the number of queens to be placed
4
1 solution
1 row <----> 2 column
2 row <----> 4 column
3 row <----> 1 column
4 row <----> 3 column
2 solution
1 row <----> 3 column
2 row <----> 1 column
3 row <----> 4 column